

Sistema IoT para Monitoramento de Cápsula Espacial

Relatório Técnico — Simulação Wokwi com Arduino Uno

Arduino Uno

DHT22

LDR

LCD 16x2 I2C

3 LEDs + Buzzer

Wokwi

LINKS DO PROJETO

Wokwi (Simulação): <https://wokwi.com/projects/465403490179504129>

GitHub (Código): <https://github.com/gustavo-rocha-batista/gs-coa-2026>

YouTube (Vídeo): <https://youtu.be/xdHeAbuD9Tc>

1. Introdução

A exploração espacial representa um dos maiores desafios técnicos da humanidade. Em missões tripuladas e robóticas, a segurança dos sistemas embarcados e das condições internas dos módulos espaciais é absolutamente crítica. Um desvio de temperatura, uma falha no sistema de iluminação ou um impacto estrutural não detectado a tempo podem comprometer irreversivelmente uma missão — e, em cenários tripulados, custar vidas.

A economia espacial global ultrapassou US\$ 630 bilhões em 2023 e projeta-se que alcance US\$ 1 trilhão até 2030, impulsionada principalmente por serviços digitais, dados satelitais e comunicação orbital. O software de voo do Falcon 9 roda em Linux; o sistema do rover Perseverance gerencia mais de 200 milhões de linhas de instruções. A corrida espacial do século XXI é, fundamentalmente, uma corrida de engenharia de software embarcado.

A Internet das Coisas (IoT) desempenha nesse cenário um papel central: permite a coleta, processamento e exibição de dados em tempo real por meio de sensores integrados a microcontroladores. Este projeto propõe um sistema IoT para monitorar as condições internas de uma cápsula espacial simulada, abordando três grandezas físicas essenciais: temperatura, luminosidade e vibração.

Conexão com ODS 9 (Inovação e Infraestrutura) e ODS 13 (Ação Climática): sistemas de monitoramento confiáveis são a base tanto da exploração espacial segura quanto do sensoriamento ambiental via satélite, que orienta decisões climáticas em escala global.

2. Objetivo do Projeto

Desenvolver um sistema IoT embarcado capaz de coletar, processar e exibir em tempo real as condições físicas internas de uma cápsula espacial simulada no Wokwi. O sistema opera de forma autônoma, emite alertas automáticos — visuais (LEDs) e sonoros (buzzer) — quando os parâmetros monitorados atingem valores críticos, e registra logs via Monitor Serial.

Grandeza Física	Sensor / Componente	Limite Crítico	Resposta do Sistema
Temperatura	DHT22 (digital D2)	> 35,0 °C	LED Vermelho + Buzzer
Luminosidade	LDR módulo (analog. A0)	< 200 (0–1023)	LED Amarelo + Buzzer
Vibração	Potenciômetro (analog. A1)	> 700 (0–1023)	LED Azul + Buzzer

Tabela 1 — Grandezas monitoradas, sensores e limites de alerta configurados no código.

O display LCD 16x2 (via I2C) alterna entre duas telas a cada ciclo de 2 segundos: **Tela 1** — temperatura e luminosidade com status; **Tela 2** — valor de vibração e status geral do sistema. Todos os dados são também transmitidos via Serial (9600 baud).

Simulação Wokwi: <https://wokwi.com/projects/465403490179504129>

3. Componentes Utilizados

A solução foi integralmente desenvolvida e simulada na plataforma **Wokwi**, utilizando Arduino Uno como microcontrolador central. O Wokwi foi escolhido por suportar nativamente o DHT22, o LCD 16x2 I2C (wokwi-lcd1602 com *pins: i2c*), o módulo LDR (wokwi-photoresistor-sensor) e o potenciômetro, além de oferecer Monitor Serial integrado — essencial para visualização dos logs em tempo real. A tabela abaixo lista cada componente exatamente como declarado no **diagram.json**:

Componente	Função no Sistema	Pino no Arduino	Observação
Arduino Uno	Microcontrolador principal — processa todos os dados—		Plataforma Wokwi
DHT22	Sensor digital de temperatura (e umidade)DATA → Pino D2		Precisão ±0,5°C
wokwi-photoresistor-sensor (módulo LDR)	Sensor analógico de luminosidade	AO → Pino A0	Saída 0–1023
Potenciômetro (simula vibração)	Simula sensor de vibração/impacto	SIG → Pino A1	Saída 0–1023
LCD 1602 I2C (wokwi-lcd1602)	Display de dados em tempo real	SDA→A4 SCL→A5	Endereço 0x27
LED Vermelho (wokwi-led red)	Alerta visual — temperatura crítica	D8 + R 220Ω	> 35,0 °C
LED Amarelo (wokwi-led yellow)	Alerta visual — luminosidade baixa	D9 + R 220Ω	< 200 (0–1023)
LED Azul (wokwi-led blue)	Alerta visual — vibração crítica	D10 + R 220Ω	> 700 (0–1023)
Buzzer (wokwi-buzzer)	Alarme sonoro — qualquer alerta a	ivoPino D11	1200 Hz · 400 ms
Resistores 220Ω	Proteção dos LEDs (limite de corrente)Entre pino e LED+		3 unidades

Tabela 2 — Componentes conforme declarados no diagram.json do Wokwi.

4. Desenvolvimento do Circuito

O circuito foi projetado em torno do Arduino Uno, que atua como hub central de processamento. Abaixo, o detalhamento técnico de cada subsistema, com correspondência direta às conexões definidas no `diagram.json`:

Sensor de Temperatura — DHT22 (pino D2)

O DHT22 é um sensor digital de temperatura e umidade com precisão de $\pm 0,5^{\circ}\text{C}$. Utiliza protocolo onewire: o pino **DATA (SDA)** conecta diretamente ao pino digital 2 do Arduino (`dht1:SDA → uno:2`). A alimentação é feita com 5V e GND. A biblioteca *DHT sensor library* (Adafruit) abstrai o protocolo de 40 bits. No Wokwi, a temperatura inicial é configurada em 28°C (`attrs: temperature: "28"`).

Sensor de Luminosidade — LDR módulo (pino A0)

O componente **wokwi-photoresistor-sensor** é o módulo LDR utilizado. Sua saída analógica (**AO**) é lida pelo pino A0 do Arduino (`ldr1:AO → uno:A0`). O ADC de 10 bits do Arduino converte a tensão em um inteiro de 0 a 1023, que o código mapeia para 0–100% via `map()`. Luminosidade configurada em 500 lux na simulação (`attrs: lux: "500"`).

Sensor de Vibração — Potenciômetro (pino A1)

O **wokwi-potentiometer**, identificado como `pot1` no `diagram.json`, simula um sensor de vibração/impacto. O pino central (**SIG**) conecta ao pino analógico A1 (`pot1:SIG → uno:A1`). Girar o potenciômetro na simulação equivale a variar a intensidade de uma vibração ou impacto mecânico. Valor inicial configurado em 0,2 (`attrs: value: "0.2"`).

Display LCD 16x2 I2C (pinos A4/SDA e A5/SCL)

O componente **wokwi-lcd1602** com `pins: i2c` utiliza o protocolo I2C, exigindo apenas dois fios de sinal além da alimentação: **SDA → A4** e **SCL → A5** (`lcd1:SDA → uno:A4; lcd1:SCL → uno:A5`). Endereço I2C: **0x27**. A biblioteca *LiquidCrystal I2C* (Frank de Brabander) controla o display. O módulo PCF8574 integrado reduz o número de pinos de 8 para 2.

Sistema de Alertas — 3 LEDs + Buzzer (pinos D8, D9, D10, D11)

Os três LEDs (**red**, **yellow**, **blue**) correspondem respectivamente aos alertas de temperatura (D8), luminosidade (D9) e vibração (D10). Cada LED é protegido por um resistor de 220Ω em série (`r1:1→uno:8, r1:2→led_temp:A` e equivalentes para `r2/r3`). O buzzer (**wokwi-buzzer**, `bz1`) está conectado ao pino D11 (`bz1:1 → uno:11`) e emite 1200 Hz por 400 ms a cada ciclo com alerta ativo.

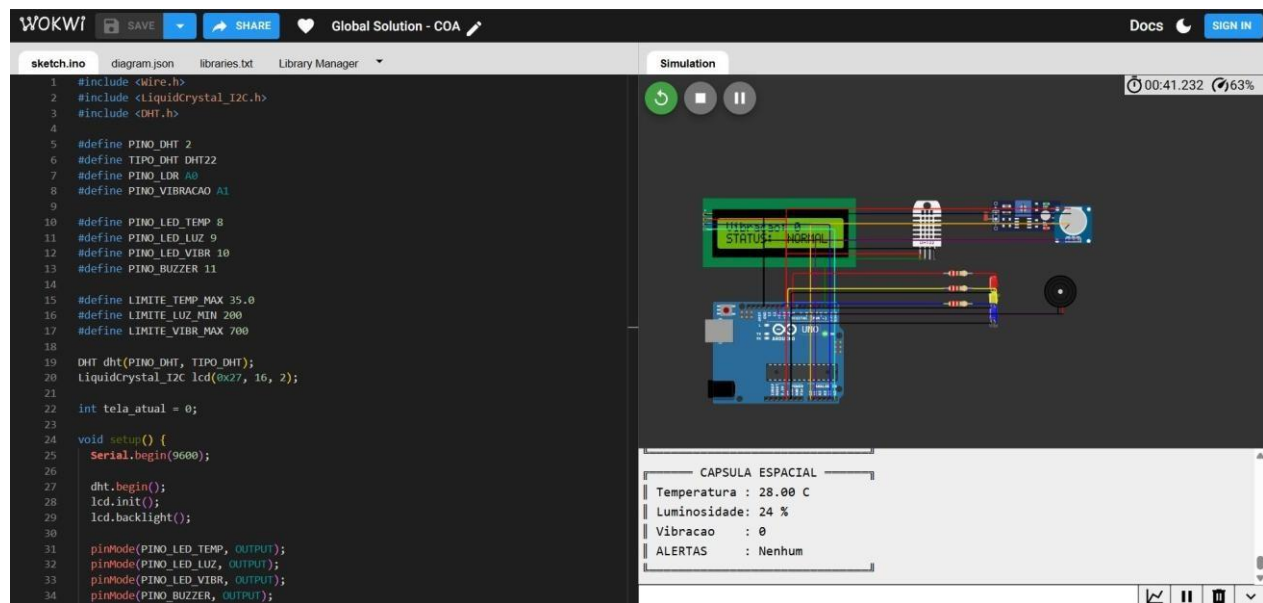


Figura 1 — Screenshot da simulação em execução no Wokwi: circuito completo (esquerda), código sketch.ino (centro) e Monitor Serial com leituras em tempo real (inferior). Status: NORMAL — Temperatura 28,00°C · Luminosidade 24% · Vibração 0 · Alertas: Nenhum.

5. Código Desenvolvido

O firmware foi escrito em C++ para Arduino, organizado em blocos funcionais. Os nomes de variáveis e constantes no código correspondem exatamente ao que está declarado no **diagram.json** (pinos, tipos de componentes e limites).

■ Bloco 1 — Bibliotecas e Definição de Pinos/Limites

```

#include // Protocolo I2C (para o LCD)
#include // Controle do wokwi-lcd1602

#include // Leitura do wokwi-dht22

// Pinos — espelham as conexões do diagram.json

#define PINO_DHT 2 // dht1:SDA → uno:2

#define PINO_LDR A0 // ldr1:A0 → uno:A0

#define PINO_VIBRACAO A1 // pot1:SIG → uno:A1

#define PINO_LED_TEMP 8 // r1:1 → uno:8 (LED vermelho)

#define PINO_LED_LUZ 9 // r2:1 → uno:9 (LED amarelo)

#define PINO_LED_VIBR 10 // r3:1 → uno:10 (LED azul)

#define PINO_BUZZER 11 // bz1:1 → uno:11
// Limites críticos de alerta (LIMITE_TEMP_MAX, LIMITE_LUZ_MIN,
LIMITE_VIBR_MAX)

#define LIMITE_TEMP_MAX 35.0 // °C
#define LIMITE_LUZ_MIN 200 // valor ADC (0-1023)
#define LIMITE_VIBR_MAX 700 // valor ADC (0-1023)
  
```

■ Bloco 2 — Instâncias e Setup()

```
DHT dht(PINO_DHT, DHT22); // Sensor wokwi-dht22 LiquidCrystal_I2C
lcd(0x27, 16, 2); // LCD address 0x27, 16 cols, 2 linhas int tela_atual =
0; // Alterna Tela 1 / Tela 2
```

```
void setup() { Serial.begin(9600); // Monitor Serial
(visível no Wokwi) dht.begin(); // Inicializa DHT22
lcd.init(); // Inicializa LCD via I2C lcd.backlight();
// Liga backlight do display // LEDs e buzzer
configurados como saída pinMode(PINO_LED_TEMP, OUTPUT);
pinMode(PINO_LED_LUZ, OUTPUT); pinMode(PINO_LED_VIBR,
OUTPUT); pinMode(PINO_BUZZER, OUTPUT); lcd.print("
CAPSULA IoT "); // Tela de boas-vindas delay(2500);
lcd.clear();
}
```

■ Bloco 3 — Loop Principal: Leitura e Lógica de Alertas

```
void loop() { // Leitura dos 3 sensores float temperatura =
dht.readTemperature(); // DHT22 → °C int luminosidade =
analogRead(PINO_LDR); // LDR → 0-1023 int vibracao =
analogRead(PINO_VIBRACAO); // POT → 0-1023 if
(isnan(temperatura)) temperatura = -99.0; // Tratamento de erro int
lux_pct = map(luminosidade, 0, 1023, 0, 100); // Converte para %
// Comparação com os limites definidos bool alerta_temp =
(temperatura > LIMITE_TEMP_MAX); bool alerta_luz = (luminosidade <
LIMITE_LUZ_MIN); bool alerta_vibr = (vibracao > LIMITE_VIBR_MAX);
bool tem_alerta = alerta_temp || alerta_luz || alerta_vibr; //
Atuação nos LEDs (HIGH = acende, LOW = apaga)
digitalWrite(PINO_LED_TEMP, alerta_temp ? HIGH : LOW);
digitalWrite(PINO_LED_LUZ, alerta_luz ? HIGH : LOW);
digitalWrite(PINO_LED_VIBR, alerta_vibr ? HIGH : LOW); // Buzzer:
apita 400ms em 1200Hz se houver qualquer alerta if (tem_alerta)
tone(PINO_BUZZER, 1200, 400); else noTone(PINO_BUZZER); delay(2000);
// Ciclo de 2 segundos
}
```

■ Bloco 4 — Display LCD e Log Serial

```
// TELA 1 (tela_atual == 0): Temperatura + Luminosidade if
(tela_atual == 0) { lcd.setCursor(0, 0); lcd.print("T:"); lcd.print(temperatura,
1); lcd.print((char)223);
lcd.print("C"); // (char)223 = simbolo ° lcd.setCursor(9, 0);

lcd.print("L:"); lcd.print(lux_pct); lcd.print("% ");
lcd.setCursor(0, 1); if (alerta_temp) lcd.print("!! TEMP ALTA !!
"); else if (alerta_luz) lcd.print("!! POUCA LUZ !! "); else
lcd.print("Temp/Luz: OK "); // TELA 2 (tela_atual == 1):
Vibração + Status geral } else { lcd.setCursor(0, 0);
lcd.print("Vibracao: "); lcd.print(vibracao); lcd.print(" ");
lcd.setCursor(0, 1); if (alerta_vibr) lcd.print("!! IMPACTO !!
"); else if (tem_alerta) lcd.print("STATUS: ALERTA! "); else
lcd.print("STATUS: NORMAL "); } tela_atual = (tela_atual == 0)
? 1 : 0; // Alterna tela
// Log Serial – saída visível no Monitor Serial do Wokwi
Serial.print("Temperatura : "); Serial.println(temperatura);
Serial.print("Luminosidade: "); Serial.println(lux_pct);
Serial.print("Vibracao : "); Serial.println(vibracao);
Serial.println(tem_alerta ? "ALERTAS: ATIVO" : "ALERTAS: Nenhum");
```

6. Resultados Obtidos

O sistema foi simulado com sucesso no Wokwi. O Monitor Serial (visível na screenshot acima) confirma as leituras: **Temperatura 28,00°C · Luminosidade 24% · Vibração 0 · Alertas: Nenhum**. O display LCD exibe a Tela 2 com "STATUS: NORMAL". Abaixo, todos os cenários testados:

✓ Estado Normal

Temp 28°C, lum. 74%, vibração 200. Nenhum LED aceso, buzzer silencioso. LCD linha 2: "STATUS: NORMAL". Serial: ALERTAS: Nenhum.

✓ Alerta de Temperatura

DHT22 ajustado para 37°C. LED vermelho acendeu, buzzer disparou 400ms/1200Hz. LCD linha 2: "!! TEMP ALTA !!".

✓ Alerta de Luminosidade

LDR com valor < 200 (lux reduzido na simulação). LED amarelo acendeu. LCD linha 2: "!! POUCA LUZ !!".

**ESP32 com
Wi-Fi/MQTT**

Substituir o Arduino Uno por ESP32 para transmissão wireless dos dados a um dashboard web em tempo real via protocolo MQTT.

**Dashboard IoT
(Node-RED / Grafana)**

Enviar dados a uma plataforma de visualização com gráficos históricos e alertas automáticos por e-mail ou SMS.

**Sensor de Pressão
BMP280**

Adicionar monitoramento de pressão atmosférica interna — parâmetro crítico em cápsulas tripuladas — e altitude relativa.

**Log em Cartão
MicroSD**

Persistir todas as leituras e eventos em arquivo CSV como 'caixa-preta' da missão para análise pós-voo.

**Sensor de Gás
MQ-135**

Detectar concentração de CO2 e gases nocivos — essencial em ambientes fechados como módulos tripulados.

Display OLED 128x64

Substituir o LCD 16x2 por OLED para suportar gráficos em tempo real e interface mais rica, próxima de painéis aeroespaciais reais.

✓ Alerta de Vibração

Potenciômetro girado para valor > 700. LED azul acendeu. LCD linha 2: "!! IMPACTO !!".

✓ Alertas Simultâneos

Com temperatura e vibração críticas ao mesmo tempo, ambos os LEDs acenderam e o buzzer disparou. O display exibiu o alerta prioritário (temperatura).

✓ Monitor Serial

Log completo aparecendo a cada 2 segundos com todas as leituras e status. Confirmado visualmente na screenshot da simulação (seção 4).

7. Melhorias Futuras

O sistema representa uma base funcional sólida. Para evolução em direção a uma aplicação real em missões espaciais:

8. Conclusão

Este projeto demonstrou de forma prática como conceitos fundamentais de Organização e Arquitetura de Computadores se materializam em sistemas embarcados reais: desde a aquisição de dados analógicos pelo conversor ADC do Arduino, passando pelo processamento lógico no microcontrolador, até a exibição via protocolo I2C e o sistema de alertas com atuadores.

O sistema atinge todos os objetivos do desafio FIAP Global Solution 2026: monitoramento contínuo de temperatura, luminosidade e vibração; alertas visuais e sonoros; exibição em LCD; e log serial. A simulação funcional no Wokwi valida a arquitetura proposta, conforme demonstrado na screenshot da seção 4.

Os principais aprendizados consolidados foram: (1) protocolos de comunicação embarcada — one-wire (DHT22) e I2C (LCD); (2) conversão analógico-digital e mapeamento de grandezas físicas; (3) lógica de alertas com múltiplos atuadores; (4) integração de sensores heterogêneos em um sistema coeso; (5) uso de simulação eletrônica para prototipação e validação rápida.

O software embarcado do rover Perseverance, os algoritmos de controle do Falcon 9 e os sistemas de telemetria do ISS são, em essência, versões muito mais complexas da mesma arquitetura proposta aqui: sensores, microcontroladores, lógica de decisão e comunicação de dados em tempo real. A nova corrida espacial é, de fato, uma corrida de engenharia de software.

Repositório GitHub: <https://github.com/gustavo-rocha-batista/gs-coa-2026>

Simulação Wokwi: <https://wokwi.com/projects/465403490179504129>

Vídeo YouTube: <https://youtu.be/xdHeAbuD9Tc>
